

AI-Based Battery Fault Prediction System: Complete Technical Documentation

1. OVERALL SYSTEM OVERVIEW

Purpose of the system: To monitor, analyze, and predict incipient faults in Lithium-ion battery packs before they escalate into critical hardware failures or thermal runaway events.

Problem being solved: Conventional Battery Management Systems (BMS) are strictly reactive. They rely on hardcoded thresholds (e.g., if Voltage > 4.2V, trigger overvoltage protection). This causes a "cliff-edge" failure scenario where the system abruptly cuts off power, potentially leaving an electric vehicle stranded or causing sudden grid storage offline events.

Why predictive fault detection is better: By applying machine learning to the continuous time-series telemetry of the battery, the system identifies the *micro-signatures* of degradation (e.g., a cell's internal resistance slowly climbing, causing minute voltage sags under load). This allows the system to predict a fault *minutes, hours, or days* before the strict hardware thresholds are breached.

Overall Architecture & Data Flow: Battery Pack (Sensors) → JBD BMS → Bluetooth Module → Raspberry Pi (Gateway/Filtering) → Data Processing Pipeline → Time-Series Transformer ML Model → Fault Prediction → Web Dashboard Visualization → Alert System

2. HARDWARE ARCHITECTURE

Battery Pack

- **Cell Voltage:** Measured at the individual series level. Critical for identifying imbalances, weak cells, and state-of-charge (SoC) drift.
- **Pack Voltage:** The aggregate series voltage. Used to determine overall system capacity and load response.
- **Current:** Monitored via a high-precision shunt resistor. Indicates the exact load demands (Charge/Discharge) driving the battery, crucial for context-aware ML inference.
- **Temperature:** Monitored via NTC thermistors. Essential for predicting thermal anomalies, as internal resistance and chemical kinetics are highly temperature-dependent.

JBD BMS (Battery Management System)

- **Data Acquisition:** Acts as the primary analog-to-digital converter (ADC), sampling cell voltages and shunt current at high frequencies.
- **Cell Monitoring & Balancing:** Monitors individual cells and applies passive balancing (bleeding excess voltage across resistors) during charging.
- **Protection Functions:** Provides the foundational Layer-0 hardware protection (MOSFET cutoff) if predictive layers fail or are bypassed.

Bluetooth Communication

- **Transmission:** Uses BLE (Bluetooth Low Energy) to broadcast BMS telemetry.
- **Packet Structure:** Data is packaged in hexadecimal payload frames consisting of headers, cell voltage blocks, temperature data, and checksums.
- **Sampling & Latency:** Broadcasts at approximately 1 Hz (1 row per second). BLE introduces minor latency (~100-300ms) but provides isolated, wireless data acquisition preventing ground loops.

Raspberry Pi (Edge Gateway)

- **Role:** Acts as the edge-compute gateway. It runs the `bms-bluetooth` service to poll the JBD BMS.
- **Processing & Dashboard:** Preprocesses the hex payloads, runs the local ML inference engine, and hosts the FastAPI backend and React/Vite dashboard, keeping the system entirely localized and independent of cloud latency.

3. COMPLETE DATA PIPELINE

Raw Data Collection

- **Sampling Interval:** 1 sample per second (1 Hz).
- **Timestamp Generation:** Precise UNIX timestamps are generated upon receipt at the Pi to ensure accurate time-series sequential alignment.
- **Data Points:** 8 to 16 individual cell voltages, pack current (Amps), and 2 to 4 NTC thermistor temperatures (°C).

Data Logging

- **CSV Structure:** Data is dumped into chronological CSV files (`timestamp, voltage, current, temperature, cell_v1... fault_label`).
- **Historical Storage:** Allows the system to look back at previous cycles to establish baseline degradation trends.

Data Cleaning

- **Missing Values:** Handled via forward-fill (carrying the last known good value) to prevent sequence breakage.
- **Noise Removal:** Sensor jitter is smoothed using digital filtering techniques (see section 4).
- **Invalid Samples:** CRC checksum failures from BLE packets are dropped entirely.

4. SIGNAL PROCESSING AND FILTERING

Why filtering is required: Raw BMS data suffers from ADC quantization noise, BLE packet dropouts, and sudden load-induced current spikes (e.g., motor inrush current) which can trick the ML model into diagnosing a false fault.

- **Moving Average Filter (MAF):** Used for smoothing overall Pack Voltage. Formula: $y[n] = (1/N) * \sum(x[n-i])$. *Advantage:* Simple and removes high-frequency noise. *Limitation:* Introduces phase lag.
- **Exponential Moving Average (EMA):** Used for Temperature tracking. Formula: $y[n] = \alpha * x[n] + (1-\alpha) * y[n-1]$. *Advantage:* Gives more weight to recent data, excellent for slow-moving thermal inertia.

- **Median Filter:** Used for Current spikes. *Advantage:* Completely ignores extreme outliers (e.g., a 1-millisecond 200A spike) without skewing the local average.
 - **Low-pass Filter:** Used for individual Cell Voltages to remove ADC jitter while preserving the actual chemical voltage sag.
-

5. FEATURE ENGINEERING

Converting raw telemetry into highly correlated ML features:

Primary Features: Raw Cell Voltages, Pack Voltage, Current, Temperature. **Derived Features:**

- **Voltage Sag (ΔV_{sag}):** $\text{Voltage_idle} - \text{Voltage_load}$. Indicates internal DCIR (Direct Current Internal Resistance).
- **Voltage Difference (ΔV_{cells}):** $\text{Max_Cell_V} - \text{Min_Cell_V}$. The primary indicator of pack imbalance.
- **Temperature Gradient (dT/dt):** Rate of temperature change over time. Rapid spikes precede thermal runaway.
- **Current Change Rate (di/dt):** Differentiates between a hard short-circuit and rapid acceleration.

Behavior Features (Operating Modes): The pipeline explicitly classifies the physical state before inference:

- **IDLE:** Current $\approx 0A$.
 - **CHARGE:** Current $> 0.5A$.
 - **CRUISE:** Current $< -0.5A$ but stable (low variance).
 - **ACCEL:** Large negative current derivative.
 - **DECEL:** Positive current transition (Regen braking). *Why this improves prediction:* A voltage drop of 0.2V is a *critical fault* if the battery is IDLE, but completely *normal* if the battery is in ACCEL mode. Providing the mode gives the ML model physical context.
-

6. TIME WINDOW CREATION

Time-series faults cannot be predicted from a single snapshot. The system converts continuous data into **Sliding Windows**.

- **Sequence Length (N):** 60 samples (representing 60 seconds of history).
 - **Sliding Concept:** As row 61 arrives, row 1 is dropped. The Transformer receives rows [2 to 61] as a unified matrix.
 - **Temporal Context:** This allows the model to see the *trajectory* of a cell's voltage, not just its current state.
-

7. MACHINE LEARNING MODEL: TIME-SERIES TRANSFORMER

Why a Transformer over alternatives?

- *Thresholds:* Cannot adapt to aging or dynamic loads.
- *Random Forest / XGBoost:* Excellent for tabular data, but fundamentally ignore sequential time dependencies. They treat row 10 and row 11 as independent events.

- **LSTM / CNN:** LSTMs suffer from vanishing gradients over long sequences and process sequentially (slow). CNNs lack deep temporal understanding.
- **Transformer Encoder:** Uses **Self-Attention** to weigh the importance of every timestamp against every other timestamp simultaneously. If a temperature spike at $t=5$ correlates with a voltage sag at $t=58$, the Attention mechanism directly maps this relationship.

8. TRANSFORMER INTERNAL WORKING

1. **Input Sequence:** The 60x15 feature matrix enters the network.
2. **Positional Encoding:** Because Transformers process data in parallel, we inject sine/cosine positional embeddings so the model knows which row came first chronologically.
3. **Multi-Head Attention:** The core engine. It calculates $\text{Attention}(Q, K, V) = \frac{\text{softmax}(Q * K^T / \sqrt{d_k}) * V}{\sqrt{d_k}}$. It looks at the current voltage drop (Query) and checks past timestamps (Keys) to see if a similar load caused it, yielding a context vector (Value).
4. **Feed Forward Network & Layer Norm:** Applies non-linear transformations (ReLU/GELU) and normalizes the data to prevent gradient collapse.
5. **Classification Head:** The final encoder output is flattened and passed through Dense layers, ending in a Softmax activation function to output fault probabilities.

9. TRAINING PROCESS

- **Dataset:** Synthetic and real-world augmented lab data covering thousands of cycles of healthy and injected-fault behavior.
- **Split:** 70% Train, 15% Validation (for hyperparameter tuning), 15% Test (for unseen evaluation).
- **Data Balancing:** SMOTE (Synthetic Minority Over-sampling) or class weighting is applied because "Normal" data heavily outweighs "Fault" data.
- **Scaling:** We use **StandardScaler** ($z = (x - \mu) / \sigma$).
 - *Why required:* Transformers use gradient descent. If Pack Voltage is 40.0 and Cell Voltage is 3.5, the network will mathematically prioritize Pack Voltage simply because the absolute number is larger. Scaling normalizes all features to a mean of 0 and variance of 1.

10. FAULT CLASSES

1. **Normal:** Battery behaving within standard physics parameters based on current load.
2. **Cell Imbalance:** One cell's SoC drifts from the pack ($\Delta V > 150\text{mV}$). *Symptom:* Reduced overall pack capacity.
3. **Weak Cell (High DCIR):** A specific cell sags heavily under load but recovers at idle. *Symptom:* Localized heating, early low-voltage cutoff.
4. **Overvoltage Risk:** A cell approaches >4.2V during charge, indicating balancing failure.
5. **Undervoltage Risk:** A cell drops perilously close to 2.5V under load.
6. **Thermal Abnormality:** Temperature rises faster than the current load justifies (high dT/dt), warning of thermal runaway.

11. PREDICTION PROCESS (INFERENCE PIPELINE)

1. **Real-Time Data:** 1 row arrives from Bluetooth.
 2. **Filtering:** MAF/Median filters remove noise.
 3. **Feature Extraction:** ΔV , mode calculation, etc., are appended.
 4. **Window Creation:** The row is pushed into a 60-row FIFO buffer.
 5. **Scaling:** The 60-row matrix is normalized using the saved training scaler.
 6. **Transformer Prediction:** Matrix passes through the frozen ONNX/PyTorch model.
 7. **Softmax Probability:** Model outputs a vector: $[0.90, 0.05, 0.03\dots]$.
 8. **Fault Class:** The `argmax` (highest probability) dictates the fault.
 9. **Dashboard:** Backend emits JSON to the frontend UI via API.
-

12. CONFIDENCE SCORE

The Softmax layer outputs a probability distribution summing to 100%.

- If the model outputs: `Normal = 92%`, `Cell Imbalance = 6%`, `Weak Cell = 2%`, the **Confidence Score** is 92%.
 - *Interpretation:* A 92% confidence means the temporal signatures in the current 60-second window closely match 92% of the spatial geometry of the "Normal" training data.
 - *Why not 100%?* Real-world sensor noise and boundary edge-cases mean data is rarely absolute. If confidence drops below 70%, the system flags the prediction as "Low Confidence" to warn engineers of anomalous, untrained behavior.
-

13. MODEL EVALUATION

- **Accuracy:** Total correct predictions / Total predictions. (Flawed metric for imbalanced datasets).
 - **Precision:** True Positives / (True Positives + False Positives). *How many of the flagged faults were actual faults?*
 - **Recall:** True Positives / (True Positives + False Negatives). *How many of the actual faults did we successfully catch?*
 - **F1 Score:** The harmonic mean of Precision and Recall.
 - *Why F1 is preferred:* In battery safety, predicting a false fault (False Positive) is annoying, but missing a real fault (False Negative) causes a fire. F1 balances these cleanly, ensuring the model doesn't just guess "Normal" 99% of the time to achieve high accuracy.
-

14. DASHBOARD ARCHITECTURE

- **Frontend:** React / Vite. Highly responsive, component-driven UI utilizing modern graphing libraries (Recharts/Chart.js) to render 60-second sliding data windows at 60fps.
 - **Backend:** FastAPI (Python). Ultra-fast, async web framework. Handles BLE stream ingestion, runs the ML inference natively in Python, and serves HTTP/WebSocket routes.
 - **Updates:** The frontend polls the backend via `fetch` at 1Hz, dynamically updating the DOM, shifting graph arrays, and pushing alerts to the Modal Overlay.
-

15. ALERT SYSTEM

- **Normal:** Green badges, silent operation.
 - **Warning (Yellow):** Triggered by anomalies like "High Deviation" or "Stale Data". Emits a soft UI beep and an unobtrusive notification.
 - **Critical Fault (Red):** Triggered by exact ML fault matches or hardware disconnects. Overrides the UI with a pulsing red border, triggers a loud, looping alarm tone, and forces a modal popup requiring explicit user acknowledgement.
-

16. SYSTEM PERFORMANCE

- **Prediction Latency:** Transformer inference takes < 50ms on a Raspberry Pi 4.
 - **Dashboard Refresh:** 1 Hz (1000ms), perfectly synced with the BMS baud rate.
 - **CPU/Memory:** Model footprint is highly optimized (< 50MB RAM). CPU utilization peaks at ~15% during inference.
 - **Real-time Performance:** The system successfully parses, filters, infers, and renders the data in under 200ms total loop time, leaving 800ms of idle overhead per second.
-

17. LIMITATIONS

- **Training Cycles:** The model relies heavily on the variance of the training data. Unseen chemical degradation patterns occurring over 5 years cannot be perfectly mapped with short-term lab data.
 - **Chemistry Lock-in:** A model trained on NMC (Lithium Nickel Manganese Cobalt) cells will output wildly inaccurate predictions if applied to LFP (Lithium Iron Phosphate) due to differing flat voltage discharge curves.
 - **Sensor Inaccuracies:** The model is at the mercy of the JBD BMS hardware. If the shunt resistor drifts due to extreme ambient heat, the feature extraction will be flawed.
-

18. FUTURE IMPROVEMENTS

- **Remaining Useful Life (RUL) & SoH:** Upgrading the system from short-term fault detection to long-term State of Health and cycle-life degradation forecasting.
 - **Edge AI / TinyML:** Compiling the Transformer model using TensorRT or TensorFlow Lite Micro to run directly on an ESP32 microcontroller attached to the battery, removing the need for a Raspberry Pi gateway.
 - **CAN Bus Integration:** Upgrading from Bluetooth to isolated CAN bus architecture for automotive-grade reliability and zero-packet-loss transmission.
-

19. JUDGE Q&A (50 EXPECTED QUESTIONS & ANSWERS)

1. **Q: Why use a Transformer instead of XGBoost?** A: *XGBoost evaluates row-by-row independently. Time-Series Transformers use Self-Attention to map the temporal relationship between a current spike at $t=1$ and a voltage sag at $t=60$, which is critical for physics-based battery behavior.*
2. **Q: Why not just use threshold-based BMS logic?** A: *Thresholds are reactive. By the time a cell hits a 4.2V threshold, it is already under stress. ML detects the micro-patterns leading up to that threshold, allowing predictive intervention.*

3. **Q: How does the model generalize to other battery packs?** A: Features are normalized (e.g., using Delta Voltage rather than absolute voltage) and scaled. However, a model trained on NMC chemistry requires fine-tuning or transfer learning to operate safely on LFP chemistry.
 4. **Q: What happens if the Bluetooth sensor drops packets?** A: The system implements forward-fill imputation for short dropouts. If the data goes stale for >3 seconds, the ML inference is paused and a CRITICAL DISCONNECT alert is thrown to prevent garbage-in, garbage-out predictions.
 5. **Q: How do you prevent False Positives during heavy acceleration?** A: We utilize Feature Engineering to explicitly define "Operating Modes" (IDLE, ACCEL, CRUISE). The Transformer learns that a 0.2V drop during ACCEL is normal, but the same drop during IDLE is a Weak Cell fault. (Note: Be prepared to expand on these themes based on the 18 sections above. Judges will focus heavily on Data Quality, Model Choice (Transformer vs LSTM), and Real-world Safety implications).
-

20. 10-MINUTE PRESENTATION SCRIPT

(0:00) Slide 1: Introduction & The Problem "Good morning judges. Today, electric vehicles and grid storage systems rely on conventional Battery Management Systems. These systems are fundamentally *reactive*. They wait for a battery to breach a critical hardware limit—like an overvoltage or thermal event—and then suddenly cut power. This leads to stranded vehicles and safety hazards. We built a system that fundamentally changes this paradigm from Reactive Protection to Predictive AI Diagnostics."

(1:30) Slide 2: System Architecture "Our system connects directly to the physical battery hardware via a JBD BMS. Using BLE, we stream raw, high-resolution telemetry—cell voltages, pack current, and multi-point temperatures—into our Edge Compute gateway. Here, the data undergoes rigorous digital signal processing to remove ADC noise and inrush current spikes, creating clean, ML-ready features."

(3:00) Slide 3: Time-Series Context "Battery degradation is not a snapshot; it is a sequential behavior over time. A voltage sag is only dangerous if you understand the current load demands over the last 60 seconds. Therefore, we buffer this telemetry into rolling 60-sample matrices. This is passed into our core inference engine."

(4:30) Slide 4: The Transformer ML Model "Rather than using standard algorithms like Random Forests, we implemented a Time-Series Transformer. Using its Multi-Head Self-Attention mechanism, the model dynamically weighs the relationship between historical loads and current voltage sags. It learns the temporal micro-signatures of battery stress—allowing it to predict Cell Imbalances or High Internal Resistance faults long before hardware limits are reached."

(6:30) Slide 5: The Dashboard & Real-Time Alerting "To interface with this, we engineered an industrial-grade React/FastAPI dashboard. It polls the model's Softmax output at 1Hz. If the model detects a pattern matching a 'Weak Cell' with 94% confidence, the UI immediately shifts state, logging the fault, displaying the exact safe limits breached, and triggering audio-visual alarms for the engineering team."

(8:00) Slide 6: Performance & Limitations "The system is highly optimized, completing the entire filtering and inference loop in under 50 milliseconds on edge hardware. While currently locked to our NMC chemistry dataset, future iterations will utilize Transfer Learning to adapt to LFP chemistries and incorporate CAN-bus for automotive-grade reliability."

(9:30) Slide 7: Conclusion "By bridging the gap between raw hardware telemetry and advanced deep learning, we have created a digital twin that doesn't just report what the battery *is* doing, but predicts what it

will do. Thank you, I am now open to your questions."